

14.模型量化精度分析

1. 量化的基本介绍

定义

模型量化是指将深度学习模型中的浮点参数和操作转换为定点表示，如FLOAT32转换为INT8等。量化能够降低内存占用，实现模型压缩和推理加速，但会造成一定程度的精度损失。

量化计算原理

以线性非对称量化为例，浮点数量化为有符号定点数的计算原理如下：

$$x_{int} = clamp(\lfloor \frac{x}{s} \rfloor + z; -2^{b-1}, 2^{b-1} - 1) \tag{6-1}$$

其中 x 为浮点数， x_{int} 为量化定点数， $\lfloor \cdot \rfloor$ 为四舍五入运算， s 为量化比例因子， z 为量化零点， b 为量化位宽，如INT8数据类型中 b 为8； $clamp$ 为截断运算，具体定义如下：

$$clamp(x; a, c) = \begin{cases} a, & x < a, \\ x, & a \leq x \leq c, \\ c, & x > c, \end{cases} \tag{6-2}$$

从定点数转换为浮点数称为反量化过程，具体定义如下：

$$x \approx \hat{x} = s(x_{int} - z) \tag{6-3}$$

设量化范围为 (q_{min}) ，截断范围为 (c_{min}) ，量化参数 s 和 z 的计算公式如下：

$$s = \frac{q_{max} - q_{min}}{c_{max} - c_{min}} = \frac{q_{max} - q_{min}}{2^b - 1} \tag{6-4}$$

$$z = c_{max} - \lfloor \frac{q_{max}}{s} \rfloor \text{ 或 } z = c_{min} - \lfloor \frac{q_{min}}{s} \rfloor \tag{6-5}$$

RKNN-Toolkit2提供两种量化方式和三种量化算法，用户可以通过`rknn.config()`中的`quantized_algorithm`和

`quantized_method`参数来选择。以下是这些量化算法和特征量化方式的介绍：

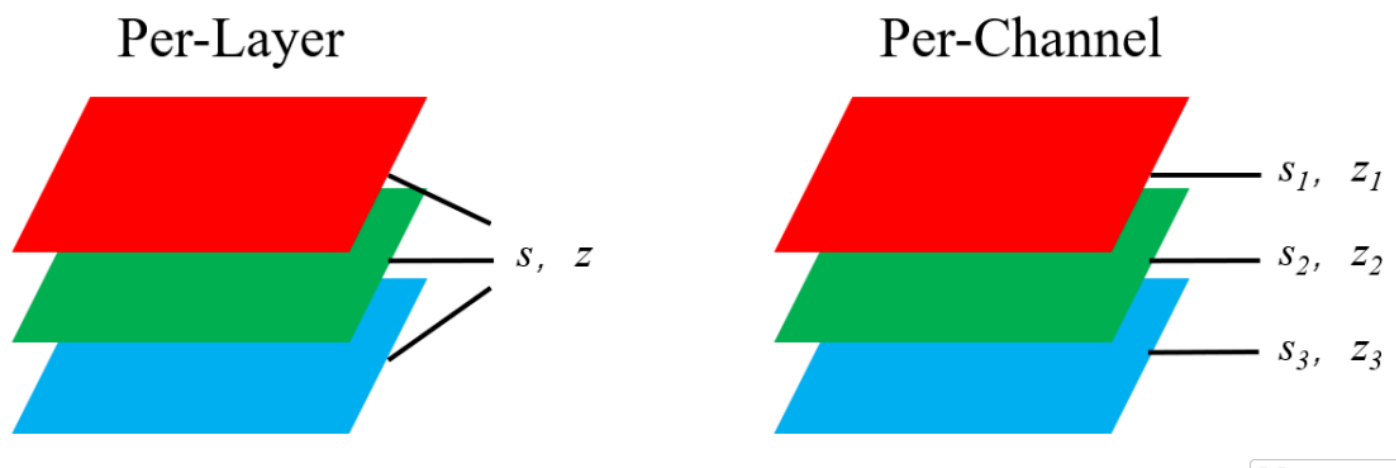
三种量化算法：

1. Normal量化算法：通过计算模型中特征（feature）的浮点数最大值和最小值来确定量化范围的最大值和最小值。算法的特点是速度较快，但是遇到特征分布不均衡时效果较差，推荐量化数据量一般为20-100张左右。

2. MMSE量化算法：通过最小化浮点数与量化反量化后浮点数的**均方误差**损失确定量化范围的最大值和最小值，能够一定程度的缓解大异常值带来的量化精度丢失问题。由于采用暴力迭代的方式，速度较慢，但通常会比normal具有更高的精度，推荐量化数据量一般为20-50张左右，用户也可以根据量化时间长短对量化数据量进行适当增减。
3. KL-Divergence量化算法：将模型特征（feature）中浮点数和定点数抽象成两个分布，通过调整不同的阈值来更新浮点数和定点数的分布，并根据KL散度衡量两个分布的相似性来确定量化范围的最大值和最小值。所用时间会比normal多一些，但比mmse会少很多，在某些场景下（feature分布不均匀时）可以得到较好的改善效果，推荐量化数据量一般为20-100张左右。

两种量化方式：

1. Layer量化方式：Layer量化方式将同一层网络的所有通道作为一个整体进行量化，所有通道共享相同的量化参数。
2. Channel量化方式：Channel量化方式将同一层网络的各个通道独立进行量化，每个通道有自己的量化参数。通常情况下，Channel量化方式比Layer量化方式具有更高的精度。



线性对称量化和线性非对称量化

这些算法到底是怎么把一个浮点数（比如 float32）变成整数（比如 int8）的？

其实，所有量化方法的核心，都是在建立浮点值和整数值之间的一种映射关系。

最常用的方式叫**线性量化**，它的**基本思想**是：用一个缩放系数（scale）和一个零点（zero point），把一段浮点范围线性地“压缩”到整数范围内。

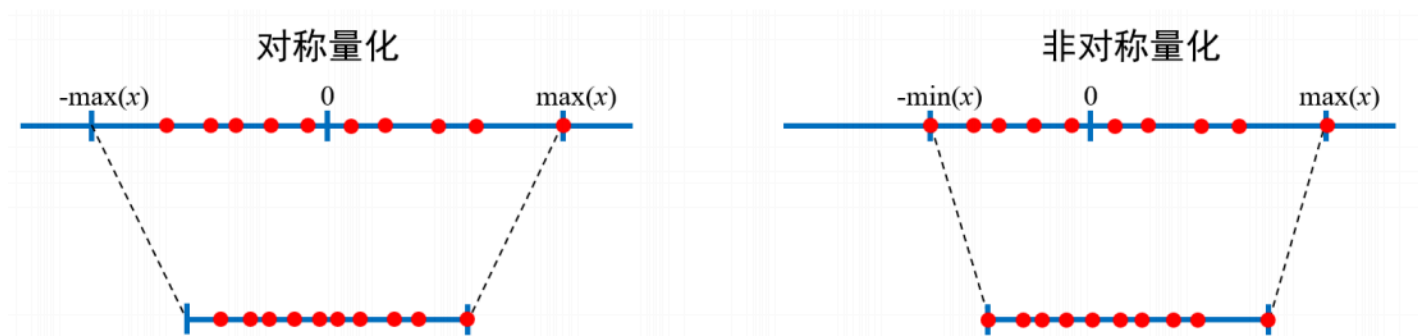
具体来说，量化过程可以简单理解为：

先把浮点数减去一个“零点”，再除以一个“缩放因子”，最后四舍五入变成整数。

反过来，如果要把整数还原成浮点数（这叫反量化），就乘上缩放因子，再加上零点。

而根据“零点”是不是固定为 0，线性量化又分为两种：

- 对称量化：强制零点为 0，适用于数据分布大致以 0 为中心的情况；
- 非对称量化：零点可以根据实际数据动态计算，能更好地适应任意分布的数据，比如 ReLU 输出这种全是正数的情况。



量化的优势：

1. 有效压缩模型文件，节省磁盘存储空间，节省成本
2. 使得模型能够更快地进行推理，可以加速模型的运行速度
3. 降低功耗
4. 节省内存和带宽

量化的缺点：精度损失！

理论上，模型推理的准确度（不过也和具体的硬件NPU有关）： $\text{float32类型} > \text{int16类型} > \text{int8类型} > \text{uint8类型}$

2. 常规量化

2.1 常规量化流程



2.2 API介绍

若量化模型推理结果有问题，可以使用 rknn.accuracy_analysis() 接口进行精度分析，查看每层算子的精度。

API	accuracy_analysis
描述	推理并产生快照，也就是dump出每一层的tensor数据。会dump出包括fp32和quant两种数据类型的快照，用于计算量化误差。 注： - 该接口只能在 build 或 hybrid_quantization_step2 之后调用。 - 如未指定target，并且原始模型应该为已量化的模型（QAT模型），则会调用失败。 - 该接口使用的量化方式与config中指定的一致。
参数	inputs: 图像（jpg / png / bmp / npy等）路径 list。

API	accuracy_analysis
	output_dir: 输出目录，所有快照都保存在该目录。默认值为'./snapshot'。 如果没有设置target，在output_dir下会输出： - simulator目录：保存整个量化模型在simulator上完整运行时每一层的结果（已转成float32）； - golden目录：保存整个浮点模型在simulator上完整跑下来时每一层的结果； - error_analysis.txt：记录simulator上量化模型逐层运行时每一层的结果与golden浮点模型逐层运行时每一层的结果的余弦距离（entire_error cosine），以及量化模型取上一层的浮点结果作为输入时，输出与浮点模型的余弦距离（single_error cosine），更详细的信息请查看error_analysis.txt文件。 如果有设置target，则在output_dir里还会多输出： - runtime目录：保存整个量化模型在NPU上完整运行时每一层的结果（已转成float32）。 - error_analysis.txt：在上述记录的内容的基础上，还会记录量化模型在simulator上逐层运行时每一层的结果与NPU上逐层运行时每一层的结果的余弦距离（entire_error cosine）等信息，更详细的信息请查看error_analysis.txt文件。
	target: 目标硬件平台，支持“rv1103”、“rv1103b”、“rv1106”、“rv1106b”、“rv1126b”、“rk3562”、“rk3566”、“rk3568”、“rk3576”和“rk3588”，默认为None。 如果设置了target，则会获取NPU运行时每一层的结果，并进行精度的分析。
	device_id: 设备编号，如果PC连接多台设备时，需要指定该参数，设备编号可以通过“list_devices”接口查看。默认值为None。

注意事项：

- 1.精度分析时可能会因为模型太大且开发板上存储容量不够导致运行失败，可在开发板终端上使用 **df-h** 命令来确认存储容量，若空间不足请删除无用文件保证 output_dir 对应分区有足够的存储空间用于保存结果文件。
- 2.通过 rknn.load_rknn() 加载RKNN模型后，因RKNN模型缺失原始模型信息，因此无法使用模型精度分析功能。
- 3.在ARM64版本中 rknn.accuracy_analysis() 无需添加device_id，并且该版本不支持adb设备连接到其他设备做精度分析。

举例如下：

代码块

```
1 # Accuracy analysis
2 ret = rknn.accuracy_analysis(inputs=['./dog_224x224.jpg'])
```

2.3 代码测试

04_accuracy_analysis

模拟器

代码块

```
1  from rknn.api import RKNN
2
3  if __name__ == '__main__':
4      # 使用RKNN方法创建RKNN对象
5      rknn = RKNN()
6
7      # 使用config接口配置要生成的RKNN对象
8      rknn.config(
9          mean_values=[123.675, 116.28, 103.53],
10         std_values=[58.395, 58.395, 58.395],
11         target_platform='rk3588'
12     )
13
14     # 使用load_xxx接口加载常用深度学习模型
15     rknn.load_pytorch(
16         model="./resnet18.pt",
17         input_size_list=[[1, 3, 224, 224]]
18     )
19
20     # 使用build接口构建RKNN模型
21     rknn.build(
22         do_quantization=True,
23         dataset='dataset.txt'
24     )
25
26     # 使用export_rknn接口导出RKNN模型
27     rknn.export_rknn(
28         export_path="./resnet18.rknn"
29     )
30
31     # 使用accuracy_analysis 接口进行模型量化精度分析
32     rknn.accuracy_analysis(
33         inputs=["space_shuttle_224.jpg"], # inputs用来表示进行推理的图像
34         output_dir="snapshot", # output_dir表示精度分析的输出目录
35         # target="rk3588", # target表示目标硬件平台
36         target=None, # target表示目标硬件平台
37         device_id=None, # device_id表示设备的编号
38     )
39     # 使用release接口释放RKNN模型
40     rknn.release()
41
```

layer_name	simulator_error			
	entire		single	
	cos	euc	cos	euc
[Input] x.3	1.00000	0.0	1.00000	0.0
[exDataConvert] x.3_int8	0.99999	2.1522	0.99999	2.1522
[Conv] input.11				
[Relu] 82	0.99991	4.4227	0.99991	4.4227
[MaxPool] input.13	0.99995	2.2783	0.99998	1.3114
[Conv] input.19				
[Relu] 120	0.99966	2.2190	0.99979	1.7615
[Conv] out.3	0.99964	4.7999	0.99996	1.5311
[Add] input.25				
[Relu] 142	0.99978	5.3705	0.99996	2.1654
[Conv] input.29				
[Relu] 169	0.99918	3.1885	0.99982	1.4863
[Conv] out.5	0.99871	9.8806	0.99993	2.2329
[Add] input.33				
[Relu] 191	0.99938	10.869	0.99996	2.8118
[Conv] input.37				
[Relu] 222	0.99859	3.2987	0.99991	0.8421
[Conv] out.7	0.99881	5.5206	0.99993	1.3496
[Conv] identity.2	0.99856	3.9809	0.99972	1.7560
[Add] input.43				
[Relu] 267	0.99886	4.8305	0.99992	1.2945
[Conv] input.47				
[Relu] 294	0.99819	2.5897	0.99987	0.6956
[Conv] out.9	0.99897	5.5916	0.99997	0.9661
[Add] input.51				
[Relu] 316	0.99862	5.5386	0.99992	1.3672
[Conv] input.55				

连板推理

代码块

```
1  from rknn.api import RKNN
2
3  if __name__ == '__main__':
4      # 使用RKNN方法创建RKNN对象
5      rknn = RKNN()
6
7      # 使用config接口配置要生成的RKNN对象
8      rknn.config(
9          mean_values=[123.675, 116.28, 103.53],
10         std_values=[58.395, 58.395, 58.395],
11         target_platform='rk3588'
12     )
13
14     # 使用load_xxx接口加载常用深度学习模型
15     rknn.load_pytorch(
16         model="./resnet18.pt",
17         input_size_list=[[1, 3, 224, 224]]
18     )
19
20     # 使用build接口构建RKNN模型
21     rknn.build(
22         do_quantization=True,
23         dataset='dataset.txt'
24     )
25
26     # 使用expoer_rknn接口导出RKNN模型
```

```

27     rknn.export_rknn(
28         export_path="./resnet18.rknn"
29     )
30
31     # 使用accuracy_analysis 接口进行模型量化精度分析
32     rknn.accuracy_analysis(
33         inputs=["space_shuttle_224.jpg"], # inputs用来表示进行推理的图像
34         output_dir="snapshot", # output_dir表示精度分析的输出目录
35         target="rk3588", # target表示目标硬件平台
36
37     )
38     # 使用release接口释放RKNN模型
39     rknn.release()
40

```

layer_name	simulator_error				runtime_error			
	entire		single		entire		single_sim	
	cos	euc	cos	euc	cos	euc	cos	euc
[Input] x.3	1.00000	0.0	1.00000	0.0				
[exDataConvert] x.3_int8	0.99999	2.1522	0.99999	2.1522				
[Conv] input.11								
[Relu] 82	0.99991	4.4227	0.99991	4.4227	0.99991	4.4429	0.99992	4.1792
[MaxPool] input.13	0.99995	2.2783	0.99998	1.3114	0.99995	2.2831	1.00000	0.0
[Conv] input.19								
[Relu] 120	0.99966	2.2190	0.99979	1.7615	0.99968	2.1540	1.00000	0.0831
[Conv] out.3	0.99964	4.7999	0.99996	1.5311				
[Add] input.25								
[Relu] 142	0.99978	5.3705	0.99996	2.1654	0.99980	5.2281	1.00000	0.2227
[Conv] input.29								
[Relu] 169	0.99918	3.1885	0.99982	1.4863	0.99925	3.0477	1.00000	0.0903
[Conv] out.5	0.99871	9.8806	0.99993	2.2329				
[Add] input.33								
[Relu] 191	0.99938	10.869	0.99996	2.8118	0.99945	10.303	1.00000	0.2685
[Conv] input.37								
[Relu] 222	0.99859	3.2987	0.99991	0.8421	0.99877	3.0882	1.00000	0.0616
[Conv] out.7	0.99881	5.5206	0.99993	1.3496	0.99877	5.6210	0.99981	2.1970
[Conv] identity.2	0.99856	3.9809	0.99972	1.7560				
[Add] input.43								
[Relu] 267	0.99886	4.8305	0.99992	1.2945	0.99890	4.7605	0.99988	1.5886
[Conv] input.47								
[Relu] 294	0.99819	2.5897	0.99987	0.6956	0.99831	2.4983	1.00000	0.0427
[Conv] out.9	0.99897	5.5916	0.99997	0.9661				
[Add] input.51								
[Relu] 316	0.99862	5.5386	0.99992	1.3672	0.99865	5.4585	1.00000	0.1088

3. 混合量化

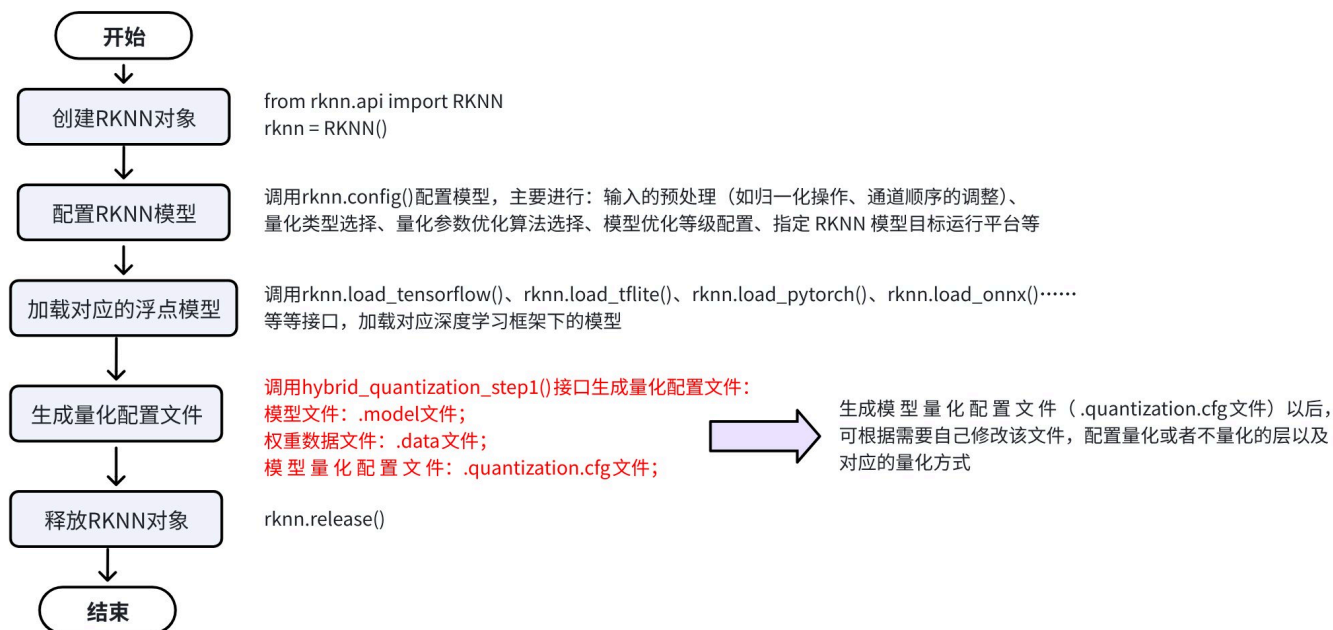
混合量化，指在网络模型中，权重和激活值可以采用不止一种量化精度类型，有多种量化精度类型，例如，某些层的权重和激活值采用uint8量化精度类型，某些层的权重和激活值采用int16量化精度类型，这里就有两种量化精度类型在整个模型中混合使用了，这种方式就叫做模型的混合量化。

目前混合量化功能支持如下用法：

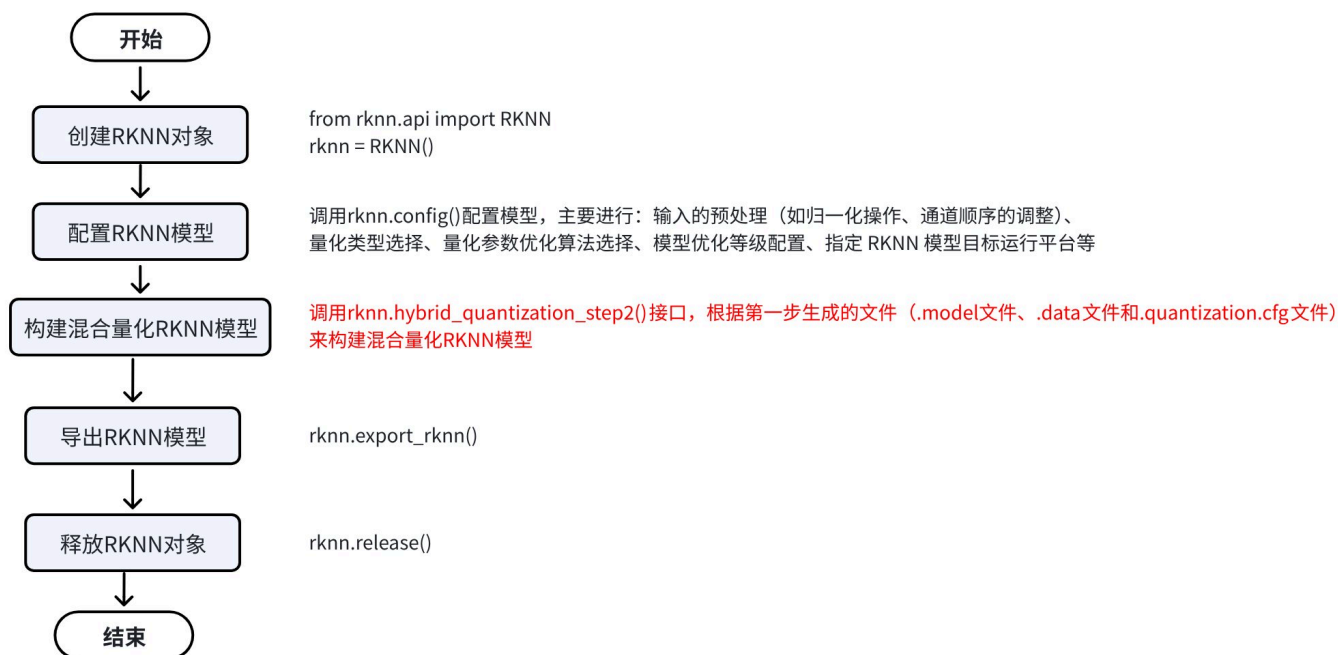
1. 将指定的量化层改成非量化层，如用 FLOAT16 进行计算。(因NPU上非量化算力较低，所以推理速度会有一定降低)。
2. 每一层的量化参数也可以进行修改。(量化参数不建议修改)

3.1 混合量化流程

混合量化第一步



混合量化第二步



3.2 API介绍

hybrid_quantization_step1

使用混合量化功能时，第一阶段调用的主要接口是hybrid_quantization_step1，用于生成临时模型文件（<model_name>.model）、数据文件（<model_name>.data）和量化配置文件（<model_name>.quantization.cfg）。接口详情如下：

API	hybrid_quantization_step1
描述	根据加载的原始模型，生成对应的临时模型文件、配置文件和量化配置文件。
参数	dataset: 见 2.4 构建RKNN模型 的dataset说明。
	rknn_batch_size: 见 2.4 构建RKNN模型 的rknn_batch_size说明。
	proposal: 产生混合量化的配置建议值。默认值为False。
	proposal_dataset_size: proposal使用的dataset的张数。默认值为1。因为proposal功能比较耗时，所以默认只使用1张，也就是dataset里的第一张。
	custom_hybrid: 用于根据用户指定的多组输入和输出名，选取混合量化对应子图。格式为[[input0_name, output0_name], [input1_name, output1_name], ...]。默认值为None。 注：输入和输出名应根据生成的临时模型文件(<model_name>.model)来选择。
返回值	0：成功。
	-1：失败。

举例如下：

代码块

```
1  # 调用hybrid_quantization_step1 产生量化配置文件
2  ret = rknn.hybrid_quantization_step1(dataset='./dataset.txt')
```

hybrid_quantization_step2

用于使用混合量化功能时生成RKNN模型，接口详情如下：

API	hybrid_quantization_step2
描述	接收临时模型文件、配置文件、量化配置文件和校正数据集作为输入，生成混合量化后的RKNN模型。
参数	model_input: hybrid_quantization_step1生成的临时模型文件（<model_name>.model）路径。 data_input: hybrid_quantization_step1生成的数据文件（<model_name>.data）路径。 model_quantization_cfg: hybrid_quantization_step1生成并经过修改后的模型量化配置文件（<model_name>.quantization.cfg）路径。
返回值	0: 成功。
	-1: 失败。

举例如下：

代码块

```

1  # Call hybrid_quantization_step2 to generate hybrid quantized RKNN model
2  ret = rknn.hybrid_quantization_step2(
3      model_input='./ssd_mobilenet_v2.model',
4      data_input='./ssd_mobilenet_v2.data',
5      model_quantization_cfg='./ssd_mobilenet_v2.quantization.cfg')

```

3.3 代码测试

05_hybrid_quantization

step1.py

代码块

```

1  from rknn.api import RKNN
2
3  if __name__ == '__main__':
4      # 使用RKNN方法创建RKNN对象
5      rknn = RKNN()
6
7      # 使用config接口配置要生成的RKNN对象
8      rknn.config(
9          mean_values=[123.675, 116.28, 103.53],
10         std_values=[58.395, 58.395, 58.395],
11         target_platform='rk3588'

```

```

12     )
13
14     # 使用load_xxx接口加载常用深度学习模型
15     rknn.load_pytorch(
16         model="./resnet18.pt",
17         input_size_list=[[1, 3, 224, 224]]
18     )
19
20     # 使用hybrid_quantization_step1接口进行混合量化的第一步
21     rknn.hybrid_quantization_step1(
22         dataset="dataset.txt", # dataset表示模型量化所需要的数据集
23         proposal=True, # 设置为True, 可以自动产生混合量化的配置建议值
24         proposal_dataset_size=1 # 图片数量
25     )
26
27     rknn.release()
28

```

```

(rknn-toolkit2) topeet@ubuntu:~/rknn/04_code/05_hybrid_quantization$ ls
dataset.txt resnet18.pt resnet18.rknn space_shuttle_224.jpg step1.py step2.py
(rknn-toolkit2) topeet@ubuntu:~/rknn/04_code/05_hybrid_quantization$ python step1.py
I rknn-toolkit2 version: 2.3.2
I Loading : 100%|          | 102/102 [00:00<00:00, 167.50it/s]
I OpFusing 0: 100%|          | 100/100 [00:00<00:00, 3572.20it/s]
I OpFusing 1 : 100%|          | 100/100 [00:00<00:00, 1808.37it/s]
I OpFusing 0 : 100%|          | 100/100 [00:00<00:00, 1299.90it/s]
I OpFusing 1 : 100%|          | 100/100 [00:00<00:00, 1227.69it/s]
I OpFusing 2 : 100%|          | 100/100 [00:00<00:00, 1162.59it/s]
I OpFusing 0 : 100%|          | 100/100 [00:00<00:00, 1098.89it/s]
I OpFusing 1 : 100%|          | 100/100 [00:00<00:00, 1044.10it/s]
I OpFusing 2 : 100%|          | 100/100 [00:00<00:00, 501.97it/s]
I GraphPreparing : 100%|          | 49/49 [00:00<00:00, 19708.56it/s]
I Quantizing : 100%|          | 49/49 [00:00<00:00, 731.84it/s]
I GraphPreparing : 100%|          | 51/51 [00:00<00:00, 18796.97it/s]
I HybridProposal Step 1/4: 100%|          | 34/34 [00:00<00:00, 588.70it/s]
I HybridProposal Step 2/4: 100%|          | 34/34 [00:02<00:00, 15.43it/s]
I HybridProposal Step 3/4: 100%|          | 34/34 [00:00<00:00, 625.42it/s]
I HybridProposal Step 4/4: 100%|          | 35/35 [00:00<00:00, 63.57it/s]
(rknn-toolkit2) topeet@ubuntu:~/rknn/04_code/05_hybrid_quantization$ ls
dataset.txt resnet18.model resnet18.quantization.cfg space_shuttle_224.jpg step2.py
resnet18.data resnet18.pt resnet18.rknn step1.py
(rknn-toolkit2) topeet@ubuntu:~/rknn/04_code/05_hybrid_quantization$

```

step2.py

代码块

```

1  from rknn.api import RKNN
2
3  if __name__ == '__main__':
4      rknn = RKNN()
5
6      # 调用hybrid_quantization_step2接口进行混合量化的第二个步骤
7      rknn.hybrid_quantization_step2(
8          model_input="resnet18.model", # model_input表示第一部生成的模型文件

```

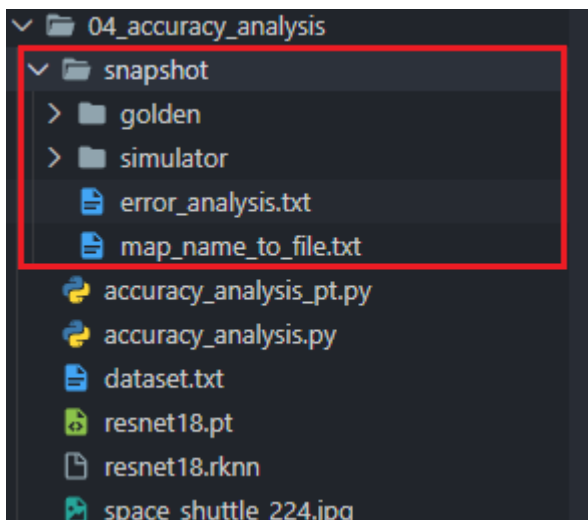
```

9         data_input="resnet18.data", # data_input表示第一步生成的配置文件
10        model_quantization_cfg="resnet18.quantization.cfg", #
        model_quantization_cfg表示第一步生成的量化配置文件
11    )
12
13    # 调用量化精度分析接口【评估RKNN模型】
14    rknn.accuracy_analysis(
15        inputs=["space_shuttle_224.jpg"],
16        output_dir="./snapshot",
17        target="rk3588"
18    )
19
20    # 调用RKNN模型导出接口导出RKNN模型
21    rknn.export_rknn(export_path="./resnet18.rknn")
22    rknn.release()

```

4. 量化结果分析

4.1 accuracy_analysis_pt.py分析



golden 子目录

- 含义：“Golden”指代未量化的原始浮点模型（如 FP32 格式的 ResNet18），作为精度分析的“黄金标准”。
- 内容：存储该模型对输入图像（如 `space_shuttle_224.jpg`）推理时，每一层的中间张量输出。
- 用途：作为基准，与量化模型的输出逐层对比，计算误差。

simulator 子目录

- 含义：模拟“Simulator”指代量化后的 RKNN 模型（如 INT8 格式），即 `build(do_quantization=True)` 生成的模型。“PC端模拟器”数据。
- 内容：存储该量化模型对同一输入图像推理时，每一层的中间张量输出。
- 用途：与 `golden` 目录中的对应层输出进行比对，分析量化引入的误差。

map_name_to_file.txt

充当了“模型结构”与“磁盘文件”之间的字典。当你需要查看某一层的具体数值时，不需要去猜测文件名，直接查这张表即可。

- 左侧 (`layer_name`): 模型计算图中的原始节点名称或内部 ID。
- 右侧 (`file_name`): 实际保存在 `snapshot/golden` 或 `snapshot/simulator` 目录下的文本文件名

error_analysis.txt

精度误差分析报告。

```

1 # simulator_error: calculate the output error of each layer of the simulator (compared to the 'golden' value).
2 #           entire: output error of each layer between 'golden' and 'simulator', these errors will accumulate layer by layer.
3 #           single: single-layer output error between 'golden' and 'simulator', can better reflect the single-layer accuracy of the simulator.
4
5 layer_name                simulator_error
6                             entire          single
7                             cos      euc      cos      euc
8 -----
9 [Input] x.3                1.00000 | 0.0      1.00000 | 0.0
10 [exDataConvert] x.3_int8  0.99999 | 2.1522  0.99999 | 2.1522
11 [Conv] input.11
12 [Relu] 82                  0.99991 | 4.4227  0.99991 | 4.4227
13 [MaxPool] input.13        0.99995 | 2.2783  0.99998 | 1.3114
14 [Conv] input.19
15 [Relu] 120                 0.99966 | 2.2190  0.99979 | 1.7615
16 [Conv] out.3              0.99964 | 4.7999  0.99996 | 1.5311
17 [Add] input.25
18 [Relu] 142                 0.99978 | 5.3705  0.99996 | 2.1654
19 [Conv] input.29
20 [Relu] 169                 0.99918 | 3.1885  0.99982 | 1.4863
21 [Conv] out.5              0.99871 | 9.8806  0.99993 | 2.2329
22 [Add] input.33
23 [Relu] 191                 0.99938 | 10.869  0.99996 | 2.8118
24 [Conv] input.37
25 [Relu] 222                 0.99859 | 3.2987  0.99991 | 0.8421
26 [Conv] out.7              0.99881 | 5.5206  0.99993 | 1.3496
27 [Conv] identity.2         0.99856 | 3.9809  0.99972 | 1.7560
28 [Add] input.43
29 [Relu] 267                 0.99886 | 4.8305  0.99992 | 1.2945
30 [Conv] input.47
31 [Relu] 294                 0.99819 | 2.5897  0.99987 | 0.6956
32 [Conv] out.9              0.99897 | 5.5916  0.99997 | 0.9661
33 [Add] input.51
34 [Relu] 316                 0.99862 | 5.5386  0.99992 | 1.3672
35 [Conv] input.55
36 [Relu] 347                 0.99865 | 2.1945  0.99990 | 0.5933
37 [Conv] out.11             0.99894 | 3.6506  0.99994 | 0.8461
38 [Conv] identity.4         0.99907 | 1.2875  0.99981 | 0.5793
39 [Add] input.61
40 [Relu] 392                 0.99888 | 2.6139  0.99993 | 0.6481
41 [Conv] input.65
42 [Relu] 419                 0.99847 | 1.2854  0.99989 | 0.3502
43 [Conv] out.13             0.99925 | 3.1385  0.99997 | 0.6257
44 [Add] input.69
45 [Relu] 441                 0.99873 | 2.7363  0.99992 | 0.7065
46 [Conv] input.8

```

Single（单层误差）—— 最重要，用于定位问题

- 定义：仅计算当前这一层输出结果的误差，不包含前面层累积的影响。
- 作用：直接反映该层算子本身的量化精度。如果某一层 `single` 误差很大，说明该层的权重或激活值在量化过程中出现了严重的信息丢失（例如数值范围过大导致截断）。

- 关注点：这是你排查问题的首选指标。

Entire（累积误差）—— 反映最终效果

- 定义：从输入层开始，一直累加到当前层的总误差。
- 作用：反映了随着网络加深，误差是如何像滚雪球一样传递和放大的。
- 现象：通常越靠后的层，`entire` 误差会越大，因为前面的微小误差会被后续的非线性操作（如 ReLU、Add）放大。

Cos vs Euc（两种计算方式）

- Cos (余弦相似度)：衡量两个向量方向的一致性。
 - 标准：越接近 1.00000 越好。
 - 解读：如果 Cos 很高（如 0.999xx），说明量化后的特征图虽然数值大小变了，但特征提取的方向是对的，模型依然能识别出物体。
- Euc (欧氏距离)：衡量两个向量在空间中的绝对距离。
 - 标准：越接近 0.0 越好。
 - 解读：这个值受数据维度（Tensor大小）影响很大。对于大尺寸的 Feature Map，即使单点误差很小，总和也可能达到几十甚至上百。不要只看绝对值，要看相对趋势。

4.2 accuracy_analysis.py

runtime 文件夹 (重点)

- 含义：“硬件运行时”数据。
- 出现原因：代码中设置了 `target="rk3588"`。
- 内容：存储量化后的模型在 真实的 RK3588 NPU 硬件环境下运行时的中间层数据。
- 作用：
 - 反映真实情况：NPU 硬件在实际计算时，可能会因为算子融合、内存对齐或特定的硬件指令集实现，产生与 PC 模拟器（simulator）微小的差异。
 - 排查硬件问题：如果 `simulator` 精度很好，但板子上跑起来精度差，就需要看 `runtime` 的数据来定位是量化策略问题还是硬件实现问题。

error_analysis.txt

```

1 # simulator_error: calculate the output error of each layer of the simulator (compared to the 'golden' value).
2 #     entire: output error of each layer between 'golden' and 'simulator', these errors will accumulate layer by layer.
3 #     single: single-layer output error between 'golden' and 'simulator', can better reflect the single-layer accuracy of the simulator.
4 # runtime_error: calculate the output error of each layer of the runtime.
5 #     entire: output error of each layer between 'golden' and 'runtime', these errors will accumulate layer by layer.
6 #     single_sim: single-layer output error between 'simulator' and 'runtime', can better reflect the single-layer accuracy of runtime.
7
8 layer_name                                simulator_error                                runtime_error
9                                     entire      single                                     entire      single_sim
10                                    cos      euc      cos      euc      cos      euc      cos      euc
11 -----
12 [Input] x.3                                1.00000 | 0.0      1.00000 | 0.0
13 [exDataConvert] x.3_int8                  0.99999 | 2.1522  0.99999 | 2.1522
14 [Conv] input.11
15 [Relu] 82                                0.99991 | 4.4227  0.99991 | 4.4227      0.99991 | 4.4429  0.99992 | 4.1792
16 [MaxPool] input.13                      0.99995 | 2.2783  0.99998 | 1.3114      0.99995 | 2.2831  1.00000 | 0.0
17 [Conv] input.19
18 [Relu] 120                              0.99966 | 2.2190  0.99979 | 1.7615      0.99968 | 2.1540  1.00000 | 0.0831
19 [Conv] out.3                            0.99964 | 4.7999  0.99996 | 1.5311
20 [Add] input.25
21 [Relu] 142                              0.99978 | 5.3705  0.99996 | 2.1654      0.99980 | 5.2281  1.00000 | 0.2227
22 [Conv] input.29
23 [Relu] 169                              0.99918 | 3.1885  0.99982 | 1.4863      0.99925 | 3.0477  1.00000 | 0.0903
24 [Conv] out.5                            0.99871 | 9.8806  0.99993 | 2.2329
25 [Add] input.33
26 [Relu] 191                              0.99938 | 10.869  0.99996 | 2.8118      0.99945 | 10.303  1.00000 | 0.2685
27 [Conv] input.37
28 [Relu] 222                              0.99859 | 3.2987  0.99991 | 0.8421      0.99877 | 3.0882  1.00000 | 0.0616
29 [Conv] out.7                            0.99881 | 5.5206  0.99993 | 1.3496      0.99877 | 5.6210  0.99981 | 2.1970
30 [Conv] identity.2                      0.99856 | 3.9809  0.99972 | 1.7560
31 [Add] input.43
32 [Relu] 267                              0.99886 | 4.8305  0.99992 | 1.2945      0.99890 | 4.7605  0.99988 | 1.5886
33 [Conv] input.47
34 [Relu] 294                              0.99819 | 2.5897  0.99987 | 0.6956      0.99831 | 2.4983  1.00000 | 0.0427
35 [Conv] out.9                            0.99897 | 5.5916  0.99997 | 0.9661
36 [Add] input.51
37 [Relu] 316                              0.99862 | 5.5386  0.99992 | 1.3672      0.99865 | 5.4585  1.00000 | 0.1088
38 [Conv] input.55
39 [Relu] 347                              0.99865 | 2.1945  0.99990 | 0.5933      0.99874 | 2.1158  1.00000 | 0.0474
40 [Conv] out.11                          0.99894 | 3.6506  0.99994 | 0.8461      0.99881 | 3.8657  0.99978 | 1.6468
41 [Conv] identity.4                      0.99907 | 1.2875  0.99981 | 0.5793
42 [Add] input.61
43 [Relu] 392                              0.99888 | 2.6139  0.99993 | 0.6481      0.99879 | 2.7144  0.99991 | 0.7402
44 [Conv] input.65
45 [Relu] 419                              0.99847 | 1.2854  0.99989 | 0.3502      0.99860 | 1.2286  1.00000 | 0.0253
46 [Conv] out.13                          0.99925 | 3.1385  0.99997 | 0.6257
47 [Add] input.69

```

4.3 resnet18.quantization.cfg

04_code > 05_hybrid_quantization >  resnet18.quantization.cfg

```
1  custom_quantize_layers: {}
2  quantize_parameters:
3    x.3:
4      qtype: asym
5      qmethod: layer
6      dtype: float32
7      min:
8        - -2.1179039478302
9      max:
10        - 2.593886375427246
11      scale: []
12      zero_point: []
13      ori_min:
14        - -2.1179039478302
15      ori_max:
16        - 2.593886375427246
17  input.11:
18    qtype: asym
19    qmethod: layer
20    dtype: int8
21    min:
22      - 0.0
23    max:
24      - 2.9629812240600586
25    scale:
26      - 0.011619534212000229
27    zero_point:
28      - -128
29    ori_min:
30      - -2.256460428237915
31    ori_max:
32      - 2.9629812240600586
```

表格

参数	你的数值示例	含义解释
qtype	asym	量化类型。 asym 代表“非对称量化”。通常与 sym（对称量化）相对。虽然该层是 FP32，但此字段可能保留了原始建议的量化方式。
qmethod	layer	量化方法。 layer 代表“层量化”，即针对该层的所有通道使用同一套缩放参数。与之相对的是 channel（通道量化），通常用于卷积核权重。
dtype	float32	数据类型。 这是核心参数。 float32 表示该层数据保持 32 位浮点格式， 不进行量化 （通常是为了保证精度，防止误差累积）。
min	-2.117...	量化后的最小值。 在 FP32 模式下，它等于 ori_min。如果是 INT8 量化，它会是类似 -128 * scale 的值。
max	2.593...	量化后的最大值。 在 FP32 模式下，它等于 ori_max。
scale	[]（空）	缩放因子。 用于 INT8 量化时的浮点数还原（ float_value = int8_value * scale ）。因为该层是 FP32，不需要转换，所以为空。
zero_point	[]（空）	零点偏移。 用于对齐 INT8 的零值。FP32 层不需要此参数。
ori_min	-2.117...	原始最小值。 在校准阶段，输入数据集中该层输出的最小浮点数值。
ori_max	2.593...	原始最大值。 在校准阶段，输入数据集中该层输出的最大浮点数值。